

Санкт-Петербургский Национальный Исследовательский Университет
Информационных Технологий, Механики и Оптики

Факультет Инфокоммуникационных Технологий
Мобильная Android разработка

Отчет по курсовой работе

Выполнил:
Борисова Э. Е
Группа
К3441

Санкт-Петербург,
2025

Реализована базовая структура Android-приложения с использованием архитектуры на основе Activity и Fragment. Создана главная активность MainActivity, которая выступает точкой входа в приложение.

Реализованный функционал

1. Главная активность

Создана MainActivity как единственная точка входа в приложение. Активность содержит Bottom Navigation для навигации между экранами.

2. Bottom Navigation

Реализована навигация с помощью BottomNavigationView, которая обеспечивает переключение между тремя основными экранами приложения: - Экран новостной ленты (NewsFragment) - Экран профиля (ProfileFragment) - Экран настроек (SettingsFragment)

3. Android Navigation Component

Навигация между экранами реализована через стандартные инструменты Android Navigation: - Создан Navigation Graph (nav_graph.xml) - Использован NavController для управления навигацией - Настроена интеграция Bottom Navigation с NavController

4. Отслеживание жизненного цикла

Во всех компонентах (Activity и Fragment) реализовано логирование ключевых событий жизненного цикла: - onCreate() / onCreateView() - onStart() / onResume() - onPause() / onStop() - onDestroy() / onDestroyView()

Все события выводятся в лог с соответствующими тегами для удобной отладки.

5. Базовые экраны

Созданы три фрагмента с базовым функционалом:

- NewsFragment - экран новостной ленты (заглушка)
- ProfileFragment - экран профиля с отображением данных пользователя
- SettingsFragment - экран настроек с переключателем темы

Ключевые особенности реализации

- Использование FragmentContainerView для размещения фрагментов

- Автоматическая привязка Bottom Navigation к NavController через `setupWithNavController()`
- Сохранение состояния навигации при повороте экрана
- Структурированный код, готовый к дальнейшему расширению

Реализована архитектура MVVM для управления состоянием приложения. Добавлены ViewModel для хранения данных профиля и настроек, обеспечено реактивное обновление UI через LiveData.

Реализованный функционал

1. ViewModel для профиля

Создан ProfileViewModel, который хранит состояние профиля пользователя: - Имя пользователя (userName) - Статус пользователя (userStatus)

Оба поля представлены как LiveData<String>, что обеспечивает реактивное обновление UI при изменении данных.

```
class ProfileViewModel : ViewModel() {  
    private val _userName =  
        MutableLiveData<String>().apply {  
            value = "Иван Иванов"  
        }  
  
    val userName: LiveData<String> = _userName  
  
    private val _userStatus =  
        MutableLiveData<String>().apply {  
            value = "В сети"  
        }  
  
    val userStatus: LiveData<String> = _userStatus  
  
    fun updateUserName(name: String) {
```

```
        _userName.value = name
    }

    fun updateUserStatus(status: String) {
        _userStatus.value = status
    }
}
```

Листинг 1 - ProfileViewModel.kt (фрагмент)

2. Экран “Профиль” с редактированием

Экран профиля обновлен для поддержки редактирования данных: - Заменены TextView на EditText для имени и статуса - При вводе текста данные автоматически сохраняются в ViewModel - Использован TextWatcher для отслеживания изменений в реальном времени - Реализована защита от циклических обновлений через флаг isUpdatingFromViewModel

3. ViewModel для настроек

Создан SettingsViewModel для управления состоянием темы приложения: - Состояние темной темы хранится как LiveData<Boolean> - ViewModel имеет область видимости Activity (activityViewModels()), что позволяет использовать один экземпляр для всех фрагментов

4. Переключатель темы

Экран настроек содержит Switch для переключения между светлой и темной темой: - Состояние переключателя синхронизировано с ViewModel через LiveData - При изменении темы обновляется AppCompatActivity для применения выбранной темы - Состояние темы сохраняется в SharedPreferences для восстановления при следующем запуске

6. Логирование жизненного цикла ViewModel

В обоих ViewModel реализовано логирование этапов жизненного цикла: - Логирование создания ViewModel в init блоке - Логирование очистки ViewModel в методе onCleared() - Логирование обновлений данных при вызове методов изменения состояния

Принципы разделения ответственности

- UI (Fragment): Отвечает только за отображение данных и обработку пользовательского ввода
- ViewModel: Управляет состоянием и бизнес-логикой, не зависит от Android фреймворка
- LiveData: Обеспечивает реактивное обновление UI при изменении данных

Реактивное обновление UI

Все обновления UI происходят через наблюдение за LiveData: - Fragment подписывается на изменения LiveData через observe() - При изменении данных в ViewModel UI автоматически обновляется - Использование viewLifecycleOwner обеспечивает правильную отписку при уничтожении View

Реализована загрузка списка сообщений из публичного API с использованием Retrofit и корутин. Данные сохраняются в локальную базу данных Room для обеспечения офлайн-доступа. Создан репозиторий как единая точка взаимодействия с данными.

Создан MessageRepository как единая точка взаимодействия ViewModel с данными: - Абстрагирует источник данных (API или локальная база) - Определяет стратегию загрузки данных в зависимости от наличия сети - Обеспечивает кэширование данных в локальной базе

```
class MessageRepository(private val context: Context)
{
    private val messageDao: MessageDao =
        MessageDatabase.getDatabase(context).messageDao()
    private val apiService = RetrofitClient.apiService

    suspend fun getMessages(): Result<List<Message>> {
        return withContext(Dispatchers.IO) {
```

```

        if (isNetworkAvailable()) {
            val response = apiService.getMessages()
            val messages =
response.take(3).mapIndexed { index, messageResponse
->
                Message(/* преобразование данных
*/)
            }
            messageDao.insertMessages(messages)
            Result.success(messages)
        } else {
            val currentMessages =
messageDao.getAllMessagesSync()
            if (currentMessages.isEmpty()) {
                Result.failure(Exception("No
network connection and no cached data"))
            } else {
                Result.success(currentMessages)
            }
        }
    }
}
}
}

```

Листинг 2 - MessageRepository.kt (основные методы)

Экран “Лента” обновлен для отображения списка сообщений: -
Использован RecyclerView для эффективного отображения больших списков -
Создан простой адаптер для отображения данных - Добавлена кнопка

“Обновить” для ручного обновления данных - Реализован индикатор загрузки и отображение ошибок

Реализована локальная база данных Room: - Создана Entity Message для хранения сообщений - Реализован DAO (MessageDao) с методами для работы с базой данных - Настроена база данных MessageDatabase с singleton паттерном - Данные автоматически сохраняются при загрузке из API

Улучшен пользовательский интерфейс с применением компонентов Material Design. Добавлена функциональность лайков сообщений, реализована фоновая синхронизация через WorkManager и система уведомлений. Добавлено управление разрешениями приложения.

Создан улучшенный адаптер для списка сообщений: - Отображение аватарки пользователя с динамической генерацией цвета - Отображение имени пользователя, заголовка и текста сообщения - Использование ListAdapter с DiffUtil для эффективных обновлений - Поддержка лайков с изменением иконки при клике

```
fun bind(message: Message) {
    userNameTextView.text = message.userName
    titleTextView.text = message.title
    bodyTextView.text = message.body

    val avatarColors = listOf(
        R.color.green_200, R.color.green_500,
        R.color.green_700
    )

    val colorIndex = message.userId % avatarColors.size
    avatarImageView.setBackgroundColor(
        ContextCompat.getColor(itemView.context,
            avatarColors[colorIndex])
    )
}
```

```
updateLikeButton(message.isLiked)

likeButton.setOnClickListener {
    val updatedMessage = message.copy(isLiked =
!message.isLiked)
    updateLikeButton(updatedMessage.isLiked)
    onLikeClick(updatedMessage)
}
}
```

Листинг 3 - MessageAdapter.kt (фрагмент bind метода)

Применены компоненты Material Design для улучшения UI:

- CardView: Каждый элемент списка обернут в CardView
- FloatingActionButton: Добавлен FAB для обновления списка сообщений с зеленым акцентным цветом
- AppBarLayout: Реализован AppBar с Toolbar для заголовка экрана
- CoordinatorLayout: Использован для координации поведения компонентов

Итоговое приложение:

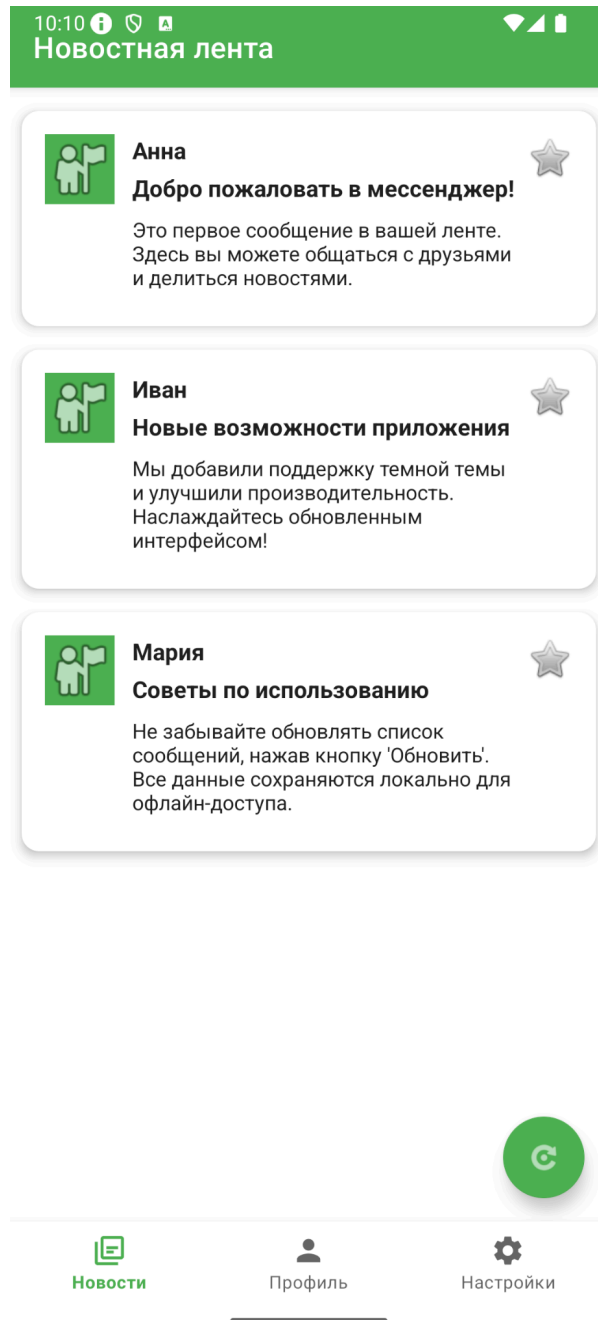


Рисунок 1 - таб новости

Профиль пользователя



Имя:

Иван Иванов

Статус:

В сети

Основная информация:

- Возраст: 25 лет
- Город: Москва
- Статус: Активен
- Последний вход: Сегодня



Новости



Профиль



Настройки

Рисунок 2 - таб профиль

Настройки

Темная тема



Другие настройки:

- Уведомления
- Безопасность
- Язык



Новости



Профиль



Настройки

Рисунок 3 - таб настройки

Вывод

В рамках работы успешно реализованы все лабораторные задания.